

Transformational Novelty with an Automata Meta-Chemistry

Alyssa M. Adams, Celia Blanco, Valerii Chirkov, Kathrin Korte, Eric Medvet, Eleni Nisioti, Sidney Pontes-Filho, Susan Stepney, Paul Templier

Cross Labs, Kyoto, Japan; Centro de Astrobiología (CSIC-INTA), Madrid, Spain; Humboldt University of Berlin, Germany; Science of Intelligence, Research Cluster of Excellence, Berlin, Germany; ITU, Copenhagen, Denmark; University of Trieste, Italy; Simula Research Laboratory, Norway; OsloMet, Norway; University of York, UK; Imperial College London, UK

Introduction

- Open-endedness - continual production of novelty - is a goal of Artificial Life
- Three levels of novelty (Banzhaf et al 2016; Stepney 2021):
 - Type 0 - variation : change to parameter values
 - Type 1 - innovation : change to the model (program *code*)
 - Type 2 - transformation : change to the metamodel (programming *language*)
- Automata chemistries - self-modifying assembly language programs - so type 1 innovation
- **Physis** (Egri-Nagy & Nehaniv, 2003) encodes the language alongside the code in the genome - both evolve - an automata *meta*-chemistry - potential for type 2 transformation
- Our work here - GPU port, reproduce original results, new results and analyses

Methodology

- Organism = tape containing language instruction header + code body (fig.1)
- Code tokens index into the language; each instruction is a sequence of micro-ops
- Mutation can change code tokens (program) and instructions (language definition)
- 128 x 128 toroidal grid = 16,384 cells
- Organisms reproduce by writing copied mutated child into a neighbouring cell
- 5 seeds x 200,000 cycles; copy-mutation rate 0.03; indel mutation rate 0.0013
- Port from original 2003 Java to Python GPU: ~20x speed-up, days to hours per run

Physis organism

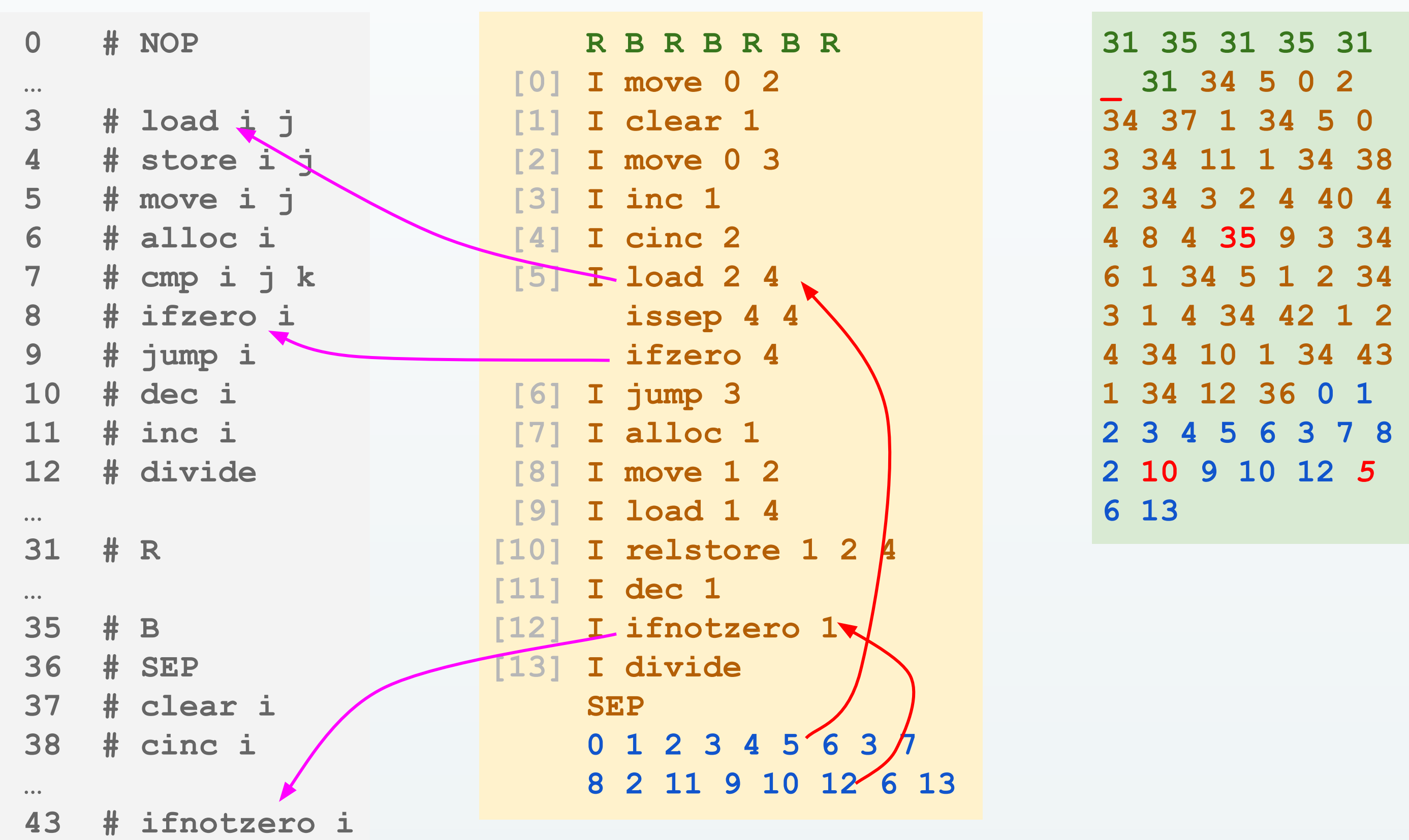


Fig. 1. Physis replicator, showing two levels of programming language.

Left: fixed set of micro-ops: same for all organisms

Middle: initial replicator parent (from Egri-Nagy & Nehaniv, 2003), parsed form

(green = register structure; orange = language definition; blue = code tokens)

Right: replicated child, unparsed form (red mutation, red italic insertion, red underline deletion)

Results

Reproduced Egri-Nagy & Nehaniv (2003) results

- Initial sparse population grows to fill the grid.
- Code body length and replication time both decrease.
- Instruction header grows slightly.

Further analysis

- Grid fills with three types of organism:
 - self-replicators (child = parent)
 - fertile (child ≠ parent)
 - infertile (no child)
- Fertile organisms can partly shield themselves behind a 'wall' of infertile children.
- Header length varies more between runs than code body length does.

Population diversity (fig. 2)

- #genomes > #executable programs > #replication speeds
- Most variation sits in tape that is never executed (junk)
- Genomic diversity measures substantially overstate phenotypic diversity

Language make-up (fig. 3)

- Many more defined vs. used instructions: the difference is 'junk language', never invoked
- Average instruction reuse > 1: code calls a compact instruction set, not a flat use-once sequence
- Average #micro-ops per instruction > 1: a genuine two-level language (micro-ops → instructions → code)

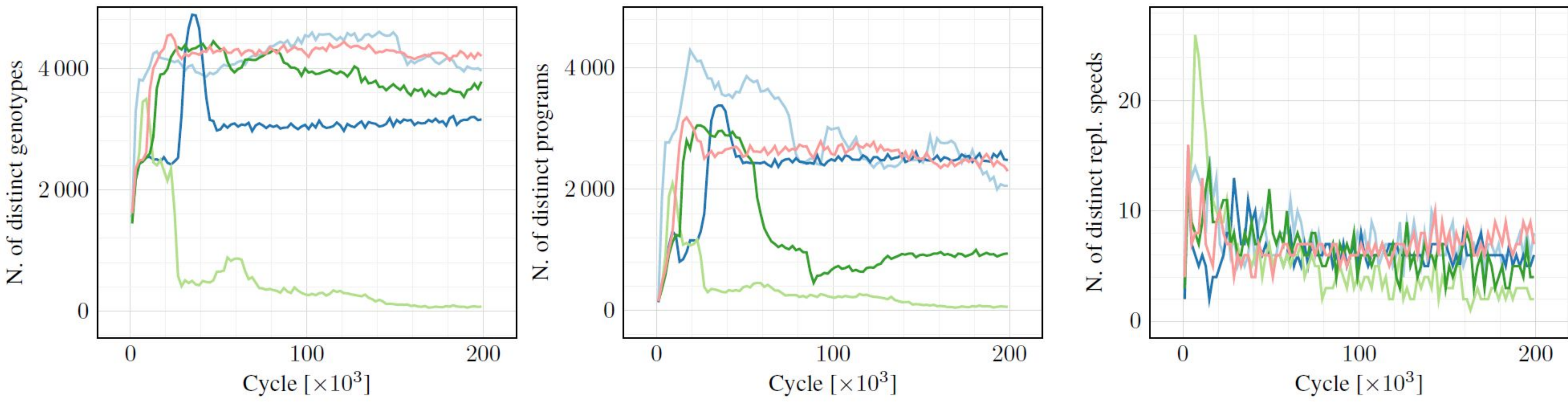


Fig. 2. Population diversity. (Colours denote independent runs)

Left: distinct self-replicating genomes (genotypes).

Middle: distinct executable programs: only executed tokens, with junk dropped.

Right: distinct replication speeds (behavioural phenotype). Diversity increases initially, then decreases or stabilises to a moderate value.

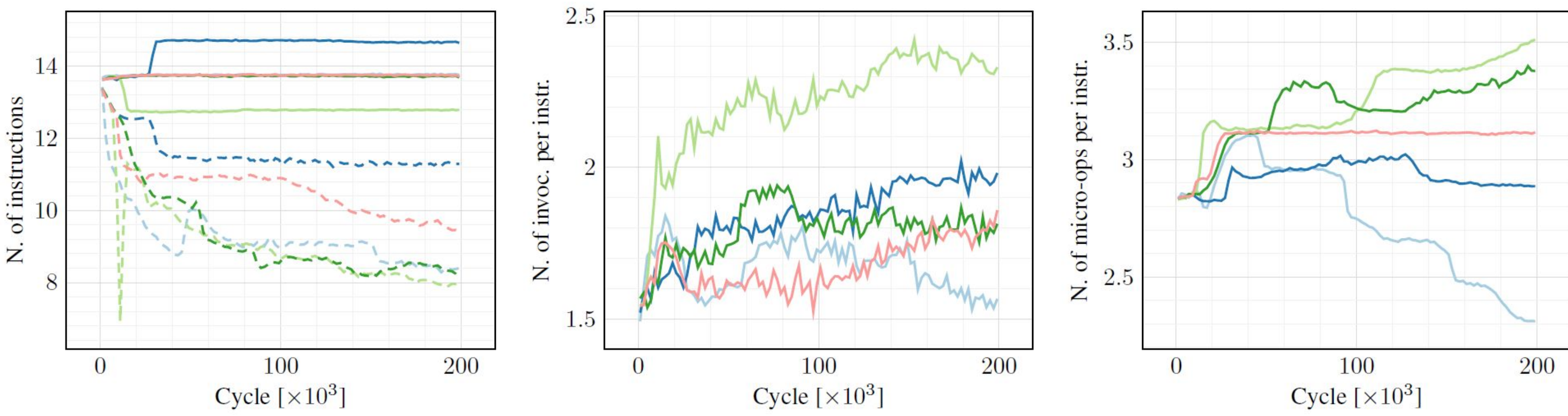


Fig. 3. Language make-up. (Colours denote independent runs)

Left: Number of instructions per genome (all vs. used).

Middle: instruction reuse / recurrence (mean invocations per used instruction).

Right: instruction complexity / hierarchy (mean micro-ops bundled per instruction).

Conclusions

- Physis (Egri-Nagy & Nehaniv, 2003) is an excellent starting point to probe level 2 transformational open-endedness: the language evolves along with the code.
- Our GPU port speedup makes larger experiment sweeps practical

Next steps

- Further code speed optimisation
- Analyse the language modifications that actually evolve
- Quantify what evolving the language buys over evolving code alone
- Add self-modification: also allow the parent to evolve, by overwriting itself
- Add direct interaction between executing organisms

Acknowledgements

- Attila Egri-Nagy, for providing the original Physis Java code.
- ALICE Workshop organisers, ITU Copenhagen, February 2026.
- Claude (Anthropic) was used to assist with some code development; authors reviewed, tested, and validated all code.

References

- Egri-Nagy, Nehaniv (2003) *ECAL 2003*, pp 238–247. Springer.
- Banzhaf *et al.* (2016). *Theory in Biosciences*, **135**(3):131–161
- Stepney. (2021). *OEE4 workshop* at ALife 2021

DATA AND CODE

